CIFP AVILÉS · CENTRO INTEGRADO DE FORMACIÓN PROFESIONAL
Departamento de Informática y Comunicaciones · Principado de Asturias

Solucionario Técnico: Bloque 1

Ficheros de Texto Plano, Charsets, Buffers y API Moderna Java NIO.2

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	**DAM 2º Curso · UT1**	**Presencial / Distancia · Java 21**	**Docente · Solución Oficial Comentada**

ALINEACIÓN CURRICULAR (RA1: MANEJO DE FICHEROS)

RA1: Manejo de Ficheros | **CE1.a (Gestión de ficheros y directorios)** | **CE1.c (Recuperar información)** | **CE1.d (Almacenar información)** | **CE1.f (Excepciones)**

Módulo: Acceso a Datos | **Ciclo**: DAM2 | **Unidad**: UT1
Hoja de Enunciados: Ver Enunciados para Alumnos
Docente: Sergio Capdevila Díez · CIFP Avilés

Solución Ejercicio 1: Validador y Normalizador de CSV (`ValidadorCSV.java`)

```
package soluciones_b1;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Locale;

public class ValidadorCSV {

    public static void main(String[] args) {
        Path rutaOrigen = Path.of("datos", "empleados_raw.csv");
        Path rutaDestino = Path.of("datos", "empleados_limpios.csv");
        Path rutaErrores = Path.of("datos", "empleados_errores.log");

        int lineasValidas = 0;
        int lineasRechazadas = 0;
        int numeroFila = 0;

        try (BufferedReader br = Files.newBufferedReader(rutaOrigen,
StandardCharsets.ISO_8859_1);
             BufferedWriter bwLimpios = Files.newBufferedWriter(rutaDestino,
```

```
StandardCharsets.UTF_8);
             BufferedWriter bwErrores = Files.newBufferedWriter(rutaErrores,
StandardCharsets.UTF_8)) {

            String linea;
            // Escribir cabecera en el fichero destino
            if ((linea = br.readLine()) != null) {
                numeroFila++;
                bwLimpios.write(linea);
                bwLimpios.newLine();
            }

            while ((linea = br.readLine()) != null) {
                numeroFila++;
                String lineaTrim = linea.trim();
                if (lineaTrim.isEmpty()) {
                    continue; // Ignorar líneas en blanco
                }

                String[] campos = linea.split(";", -1);   // -1: conserva los campos
vacíos del final
                if (campos.length != 5) {
                    bwErrores.write(String.format("[LÍNEA %d] Error: Número de campos
incorrecto (%d esperados: 5) -> Contenido: %s%n",
                            numeroFila, campos.length, linea));
                    lineasRechazadas++;
                    continue;
                }

                String dni = campos[0].trim();
                String nombre = campos[1].trim();
                String departamento = campos[2].trim();
                String salarioStr = campos[3].trim();
                String fechaAlta = campos[4].trim();

                if (dni.isBlank()) {
                    bwErrores.write(String.format("[LÍNEA %d] Error: DNI vacío ->
Contenido: %s%n", numeroFila, linea));
                    lineasRechazadas++;
                    continue;
                }

                double salario;
                try {
                    salario = Double.parseDouble(salarioStr);
                    if (!Double.isFinite(salario) || salario <= 0) {   // rechaza
también NaN e Infinity
                        throw new NumberFormatException("Salario negativo o cero");
                    }
                } catch (NumberFormatException e) {
                    bwErrores.write(String.format("[LÍNEA %d] Error: Salario inválido
('%s') -> Contenido: %s%n",
                            numeroFila, salarioStr, linea));
                    lineasRechazadas++;
                    continue;
                }

                // Si supera las validaciones, escribir en el fichero normalizado
                // Locale.US: el CSV lleva punto decimal para que otro programa pueda
leerlo con parseDouble
                bwLimpios.write(String.format(Locale.US, "%s;%s;%s;%.2f;%s", dni,
```

```
nombre, departamento, salario, fechaAlta));
                bwLimpios.newLine();
                lineasValidas++;
            }

            System.out.printf("✅ Proceso completado: %d válidos, %d rechazados.%n",
lineasValidas, lineasRechazadas);

        } catch (IOException e) {
            System.err.println("Error crítico de E/S: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola:

```
✅ Proceso completado: 2 válidos, 3 rechazados.
```

`datos/empleados_limpios.csv` (UTF-8, punto decimal):

```
DNI;NOMBRE;DEPARTAMENTO;SALARIO;FECHA_ALTA
12345678A;García Pérez, María;Administración;2450.50;2024-01-15
34567890D;Núñez Rivas, Carlos;Logística y Almacén;2100.75;2022-06-10
```

`datos/empleados_errores.log`:

```
[LÍNEA 3] Error: Salario inválido ('invalid_salary') -> Contenido: 87654321B;Muñoz
Cañete, José;Diseño & Marketing;invalid_salary;2023-11-20
[LÍNEA 4] Error: Número de campos incorrecto (4 esperados: 5) -> Contenido:
23456789C;Álvarez Díaz, Lucía;I+D;3100.00
[LÍNEA 5] Error: DNI vacío -> Contenido: ;López Vega, Ana;Ventas;1900.00;2024-03-01
```

> **Notas para el aula:** `split(";", -1)` conserva los campos vacíos del final (sin `-1`, `"a;b;;"` daría 2 campos). `Double.parseDouble("NaN")` no lanza excepción y `NaN <= 0` es falso: por eso se comprueba `Double.isFinite(salario)`. El salario se escribe con `Locale.US` porque es un CSV que leerá otro programa.

Solución Ejercicio 2: Generador de Informes de Ventas (`GeneradorInforme.java`)

```
package soluciones_b1;

import java.io.*;
import java.nio.charset.StandardCharsets;
```

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Locale;

public class GeneradorInforme {

    public static void main(String[] args) {
        Path entrada = Path.of("datos", "ventas_diarias.txt");
        File salida = new File("datos/informe_ventas.txt");

        double totalFacturacion = 0;
        int totalUnidades = 0;
        int numTiendas = 0;
        String tiendaLider = "";
        double maxVentas = -1;
        Locale es = Locale.forLanguageTag("es-ES");   // informe para personas: coma
decimal en cualquier equipo

        try (BufferedReader br = Files.newBufferedReader(entrada,
StandardCharsets.UTF_8);
             PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter(salida,
StandardCharsets.UTF_8)))) {


pw.println("================================================================================
===");
            pw.println("                INFORME DIARIO DE LIQUIDACIÓN DE TIENDAS
");
            pw.println("                     CIFP AVILÉS · RED DE VENTAS
");

pw.println("================================================================================
===");
            pw.printf(es, "%-6s %-18s %14s %8s %14s%n", "ID", "CIUDAD", "IMPORTE NETO",
"UNIDADES", "TICKET MEDIO");
            pw.println("----------------------------------------------------------------------
---------------");

            String linea;
            while ((linea = br.readLine()) != null) {
                if (linea.isBlank() || linea.startsWith("#")) continue;

                String[] p = linea.split("\\|");          // | es especial en una regex:
hay que escaparlo
                String id = p[0].trim();
                String ciudad = p[1].trim();
                double importe = Double.parseDouble(p[2].trim());
                int unidades = Integer.parseInt(p[3].trim());

                double ticketMedio = unidades > 0 ? (importe / unidades) : 0;
                totalFacturacion += importe;
                totalUnidades += unidades;
                numTiendas++;

                if (importe > maxVentas) {
                    maxVentas = importe;
                    tiendaLider = ciudad;
                }

                pw.printf(es, "%-6s %-18s %12.2f € %8d %12.2f €%n",
                        id, ciudad, importe, unidades, ticketMedio);
```

```
            }

            double mediaPorTienda = numTiendas > 0 ? (totalFacturacion / numTiendas) :
0;


pw.println("================================================================================
===");
            pw.printf(es, "TOTALES: %d Tiendas | %d Unidades Vendidas | Facturación
Total: %10.2f €%n",
                    numTiendas, totalUnidades, totalFacturacion);
            pw.printf(es, "PROMEDIO POR TIENDA: %10.2f € | TIENDA LÍDER: %s (%.2f €)%n",
                    mediaPorTienda, tiendaLider, maxVentas);

pw.println("================================================================================
===");

            System.out.println("✅ Informe generado con éxito en: " + salida.getPath());

        } catch (IOException e) {
            System.err.println("Error procesando ventas: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola: `✅ Informe generado con éxito en: datos/informe_ventas.txt`

`datos/informe_ventas.txt` (con `Locale` es-ES fijado: sale igual en cualquier equipo):

```
================================================================================
                INFORME DIARIO DE LIQUIDACIÓN DE TIENDAS
                    CIFP AVILÉS · RED DE VENTAS
================================================================================
ID     CIUDAD              IMPORTE NETO UNIDADES   TICKET MEDIO
--------------------------------------------------------------------------------
T01    Avilés Centro          14520,80 €      142       102,26 €
T02    Gijón Puerto           23100,50 €      215       107,44 €
T03    Oviedo Catedral        18940,25 €      180       105,22 €
T04    Llanes Costa            8450,00 €       94        89,89 €
================================================================================
TOTALES: 4 Tiendas | 631 Unidades Vendidas | Facturación Total:   65011,55 €
PROMEDIO POR TIENDA:   16252,89 € | TIENDA LÍDER: Gijón Puerto (23100,50 €)
================================================================================
```

> **Notas para el aula:** `split("|")` partiría la línea carácter a carácter, porque `|` es un operador de las expresiones regulares: hay que escribir `split("\\|")`. La cabecera usa los mismos anchos que las filas (`%-6s %-18s` y 14 huecos para cada importe con su `€`).

Solución Ejercicio 3: Analizador Forense con Java NIO.2 y Streams (`LogAnalyzerNIO.java`)

```
package soluciones_b1;

import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Iterator;
import java.util.Locale;
import java.util.Map;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class LogAnalyzerNIO {

    public static void main(String[] args) {
        Path logPath = Path.of("datos", "access.log");
        Path alertasPath = Path.of("datos", "peticiones_sospechosas.txt");

        try {
            // 1. Total de peticiones (Stream directo sin almacenar en memoria)
            try (Stream<String> lineas = Files.lines(logPath, StandardCharsets.UTF_8)) {
                long total = lineas.count();
                System.out.println("1. Total de peticiones auditadas: " + total);
            }

            // 2. Filtrado de HTTP >= 400 y volcado según llegan (iterator + while: sin
List intermedia)
            try (Stream<String> lineas = Files.lines(logPath, StandardCharsets.UTF_8);
                 PrintWriter pw = new PrintWriter(Files.newBufferedWriter(alertasPath,
StandardCharsets.UTF_8))) {

                Iterator<String> it = lineas.filter(l -> {
                    String[] partes = l.split(" ");
                    if (partes.length >= 9) {
                        try {
                            int status = Integer.parseInt(partes[partes.length - 2]);
                            return status >= 400;
                        } catch (NumberFormatException ignored) {}
                    }
                    return false;
                }).iterator();
                long errores = 0;
                while (it.hasNext()) {          // se lee y se escribe línea a línea
                    pw.println(it.next());
                    errores++;
                }

                System.out.println("2. Peticiones de error (>=400) registradas en
alertas: " + errores);
            }

            // 3. Suma de bytes transmitidos por respuestas 200 OK
            try (Stream<String> lineas = Files.lines(logPath, StandardCharsets.UTF_8)) {
```

```
                long bytesTotales = lineas
                        .filter(l -> l.contains("\" 200 "))
                        .mapToLong(l -> {
                            String[] partes = l.split(" ");
                            try {
                                return Long.parseLong(partes[partes.length - 1]);
                            } catch (Exception e) {
                                return 0L;
                            }
                        })
                        .sum();

                System.out.printf(Locale.forLanguageTag("es-ES"), "3. Bytes totales
transferidos con éxito (200 OK): %,d bytes (%.2f MB)%n",
                        bytesTotales, (bytesTotales / (1024.0 * 1024.0)));
            }

            // 4. Top 3 Direcciones IP más activas (groupingBy + limit)
            try (Stream<String> lineas = Files.lines(logPath, StandardCharsets.UTF_8)) {
                System.out.println("4. Top 3 Direcciones IP más activas:");
                lineas.map(l -> l.split(" ")[0].trim())
                      .collect(Collectors.groupingBy(ip -> ip, Collectors.counting()))
                      .entrySet().stream()
                      .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                              .thenComparing(Map.Entry.comparingByKey()))   // desempate
por IP
                      .limit(3)
                      .forEach(entry -> System.out.printf("   - IP: %-15s -> %d
peticiones%n", entry.getKey(), entry.getValue()));
            }

        } catch (IOException e) {
            System.err.println("Error en lectura NIO.2: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola:

```
1. Total de peticiones auditadas: 5
2. Peticiones de error (>=400) registradas en alertas: 3
3. Bytes totales transferidos con éxito (200 OK): 22.970 bytes (0,02 MB)
4. Top 3 Direcciones IP más activas:
   - IP: 192.168.1.105   -> 2 peticiones
   - IP: 10.0.0.15       -> 1 peticiones
   - IP: 10.0.0.88       -> 1 peticiones
```

`datos/peticiones_sospechosas.txt`:

```
10.0.0.15 - - [17/Sep/2026:10:14:23] "POST /login HTTP/1.1" 401 512
172.16.0.4 - - [17/Sep/2026:10:14:25] "GET /admin/db.php HTTP/1.1" 404 1204
10.0.0.88 - - [17/Sep/2026:10:14:35] "POST /checkout HTTP/1.1" 500 850
```

> **Notas para el aula:** ninguna consulta guarda el fichero en una `List`: `count()`, `mapToLong(...).sum()` y `groupingBy` trabajan en streaming, y las líneas sospechosas se escriben según llegan con `iterator()` + `while` (no con `peek()`, que es para depuración y `count()` puede no ejecutarlo). El mapa de IPs sí está en memoria, pero tiene una entrada por IP, no por línea. Tres IPs empatan a 1 petición: `thenComparing(Map.Entry.comparingByKey())` hace que el top 3 sea siempre el mismo.

Solución Ejercicio 4: Organizador de la Carpeta de Descargas (`OrganizadorDescargas.java`)

```
package soluciones_b1;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.util.Arrays;
import java.util.Map;
import java.util.TreeMap;

public class OrganizadorDescargas {

    private static final Path DESCARGAS = Path.of("datos", "descargas");

    /** Crea la carpeta de prueba con sus 8 ficheros (solo la primera vez). */
    static void prepararCarpeta() throws IOException {
        if (Files.exists(DESCARGAS)) return;                    // ya preparada en una
ejecución anterior
        Files.createDirectories(DESCARGAS);                     // crea datos y
descargas
        String[][] ficheros = {
            {"apuntes_UT1.pdf", "PDF de apuntes UT1"},
            {"factura_0925.PDF", "PDF factura septiembre"},
            {"foto_aula.jpg", "JPG foto del aula"},
            {"logo.png", "PNG logo"},
            {"notas.txt", "Notas sueltas"},
            {"README", "Leeme sin extension"},
            {"captura.JPG", "JPG captura pantalla"},
            {"datos.backup.txt", "Copia de datos"}
        };
        for (String[] f : ficheros) {
            Files.writeString(DESCARGAS.resolve(f[0]), f[1]);     // UTF-8; todo ASCII:
1 byte por carácter
        }
    }

    /** Extensión en minúsculas (lo que va tras el último punto) o "sin_extension". */
    static String extension(String nombre) {
        int punto = nombre.lastIndexOf('.');
        return punto > 0 ? nombre.substring(punto + 1).toLowerCase() : "sin_extension";
    }
```

```
    public static void main(String[] args) {
        try {
            prepararCarpeta();

            File[] contenido = DESCARGAS.toFile().listFiles();
            // listFiles() no garantiza ningún orden: ordenamos por nombre sin
distinguir mayúsculas
            Arrays.sort(contenido, (a, b) ->
a.getName().compareToIgnoreCase(b.getName()));
            Map<String, Integer> ficherosPorCarpeta = new TreeMap<>();
            Map<String, Long> bytesPorCarpeta = new TreeMap<>();

            System.out.println("📂 Organizando " + DESCARGAS + " (" + contenido.length +
" elementos)");
            for (File f : contenido) {
                if (f.isDirectory()) continue;                    // las carpetas ya
organizadas no se tocan

                String ext = extension(f.getName());
                Path carpeta = DESCARGAS.resolve(ext);
                Files.createDirectories(carpeta);                 // no falla si ya
existe
                Path destino = carpeta.resolve(f.getName());
                long tam = Files.size(f.toPath());
                Files.move(f.toPath(), destino, StandardCopyOption.REPLACE_EXISTING);

                System.out.printf("  %-18s -> %s/%n", f.getName(), ext);
                ficherosPorCarpeta.put(ext, ficherosPorCarpeta.getOrDefault(ext, 0) +
1);
                bytesPorCarpeta.put(ext, bytesPorCarpeta.getOrDefault(ext, 0L) + tam);
            }

            if (ficherosPorCarpeta.isEmpty()) {
                System.out.println("✅ No hay ficheros sueltos: la carpeta ya estaba
organizada.");
                return;
            }
            System.out.println("Resumen por carpeta:");
            for (String ext : ficherosPorCarpeta.keySet()) {
                System.out.printf("  %-14s %2d fichero(s) %5d bytes%n",
                        ext + "/", ficherosPorCarpeta.get(ext),
bytesPorCarpeta.get(ext));
            }
        } catch (IOException e) {
            System.err.println("Error organizando la carpeta: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Primera ejecución:

```
📂 Organizando datos/descargas (8 elementos)
  apuntes_UT1.pdf    -> pdf/
  captura.JPG        -> jpg/
  datos.backup.txt   -> txt/
  factura_0925.PDF   -> pdf/
  foto_aula.jpg      -> jpg/
  logo.png           -> png/
  notas.txt          -> txt/
  README             -> sin_extension/
Resumen por carpeta:
  jpg/            2 fichero(s)    37 bytes
  pdf/            2 fichero(s)    40 bytes
  png/            1 fichero(s)     8 bytes
  sin_extension/  1 fichero(s)    19 bytes
  txt/            2 fichero(s)    27 bytes
```

Segunda ejecución (la carpeta ya existe y solo contiene subcarpetas):

```
📂 Organizando datos/descargas (5 elementos)
✅ No hay ficheros sueltos: la carpeta ya estaba organizada.
```

Estructura final de `datos/descargas`:

```
datos/descargas/
├── jpg/
│   ├── captura.JPG  (20 B)
│   └── foto_aula.jpg  (17 B)
├── pdf/
│   ├── apuntes_UT1.pdf  (18 B)
│   └── factura_0925.PDF  (22 B)
├── png/
│   └── logo.png  (8 B)
├── sin_extension/
│   └── README  (19 B)
└── txt/
    ├── datos.backup.txt  (14 B)
    └── notas.txt  (13 B)
```

> **Notas para el aula:** `listFiles()` devuelve el contenido en el orden que dé el sistema de ficheros (distinto en Windows y Linux), por eso se ordena. `Arrays.sort(contenido)` a secas tampoco sirve: en Linux pondría `README` delante de `apuntes_UT1.pdf` (mayúsculas primero) y en Windows no. El tamaño se mide **antes** del `move`: después, la ruta original ya no existe (`NoSuchFileException`). `lastIndexOf('.')` y no `indexOf`: `datos.backup.txt` es un `.txt`. La condición `punto > 0` deja también los ocultos tipo `.gitignore` en `sin_extension`.

Solución Ejercicio 5: Boletín de Notas con `Scanner` y `PrintWriter` (`BoletinNotas.java`)

```
package soluciones_b1;

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.util.Locale;
import java.util.NoSuchElementException;
import java.util.Scanner;

public class BoletinNotas {

    static final String[] MODULOS = {"AD", "DI", "PMDM", "PSP", "SGE"};

    static String calificacion(double media) {
        if (media >= 9) return "Sobresaliente";
        if (media >= 7) return "Notable";
        if (media >= 6) return "Bien";
        if (media >= 5) return "Suficiente";
        return "Insuficiente";
    }

    public static void main(String[] args) {
        Locale es = Locale.forLanguageTag("es-ES");
        File entrada = new File("datos/notas_2DAM.csv");
        File salida = new File("datos/boletin_2DAM.txt");

        double[] sumaModulo = new double[MODULOS.length];   // para la media del grupo
por módulo
        int alumnosValidos = 0;
        int numLinea = 1;                                   // la cabecera es la línea
1

        try (Scanner lector = new Scanner(entrada, StandardCharsets.UTF_8);
             PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter(salida,
StandardCharsets.UTF_8)))) {

            lector.nextLine();                              // saltamos la cabecera
            pw.println("BOLETÍN DE CALIFICACIONES · 2º DAM · 1.ª EVALUACIÓN");
            pw.println("=".repeat(76));
            pw.printf(es, "%-24s", "ALUMNO");
            for (String m : MODULOS) pw.printf(es, "%6s", m);
            pw.printf(es, "%7s  %s%n", "MEDIA", "CALIFICACIÓN");
            pw.println("-".repeat(76));

            while (lector.hasNextLine()) {
                String linea = lector.nextLine();
                numLinea++;
                // Un Scanner por línea: el salto de línea no es un ';' (error habitual
del 3.3)
                try (Scanner campos = new Scanner(linea)) {
                    campos.useDelimiter(";");
                    campos.useLocale(es);                   // las notas vienen con
coma decimal: 7,5
                    String alumno = campos.next();
                    double[] notas = new double[MODULOS.length];
```

```
                    for (int i = 0; i < notas.length; i++) {
                        notas[i] = campos.nextDouble();       // "NP" ->
InputMismatchException
                    }                                         // campo que falta ->
NoSuchElementException

                    double suma = 0;
                    pw.printf(es, "%-24s", alumno);
                    for (int i = 0; i < notas.length; i++) {
                        pw.printf(es, "%6.2f", notas[i]);
                        suma += notas[i];
                        sumaModulo[i] += notas[i];
                    }
                    double media = suma / notas.length;
                    pw.printf(es, "%7.2f  %s%n", media, calificacion(media));
                    alumnosValidos++;
                } catch (NoSuchElementException e) {          // cubre también
InputMismatchException
                    System.out.printf("⚠️ Línea %d descartada (nota no numérica o
incompleta): %s%n", numLinea, linea);
                }
            }

            pw.println("-".repeat(76));
            pw.printf(es, "%-24s", "MEDIA DEL GRUPO");
            for (double s : sumaModulo) pw.printf(es, "%6.2f", s / alumnosValidos);
            pw.println();
            pw.printf(es, "Alumnado evaluado: %d%n", alumnosValidos);

            if (pw.checkError()) {                            // PrintWriter no lanza
IOException al escribir
                System.err.println("Hubo un error al escribir el boletín.");
            }
            System.out.printf("✅ Boletín generado en %s: %d alumnos evaluados.%n",
salida.getPath(), alumnosValidos);

        } catch (IOException e) {
            System.err.println("Error de E/S: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola:

```
⚠️ Línea 5 descartada (nota no numérica o incompleta): Díaz Prieto, Hugo;6;NP;7;5,5;6
⚠️ Línea 6 descartada (nota no numérica o incompleta): Fernández Gil, Sara;8;7,5;8,5;9;
✅ Boletín generado en datos/boletin_2DAM.txt: 4 alumnos evaluados.
```

`datos/boletin_2DAM.txt` (con `Locale` es-ES fijado: sale igual en cualquier equipo):

```
BOLETÍN DE CALIFICACIONES · 2º DAM · 1.ª EVALUACIÓN
================================================================================
ALUMNO                      AD    DI  PMDM   PSP   SGE  MEDIA  CALIFICACIÓN
--------------------------------------------------------------------------------
Álvarez Díaz, Lucía       7,50  8,00  6,25  9,00  7,00   7,55  Notable
Barrio Sánchez, Iván      4,50  5,00  3,00  6,50  5,00   4,80  Insuficiente
Cuesta Mier, Nerea       10,00  9,50  9,00  8,75  9,00   9,25  Sobresaliente
García Llano, Pablo       5,00  6,50  5,25  5,00  4,75   5,30  Suficiente
--------------------------------------------------------------------------------
MEDIA DEL GRUPO           6,75  7,25  5,88  7,31  6,44
Alumnado evaluado: 4
```

> **Notas para el aula:** sin `useLocale(es)` el resultado depende del equipo: con un sistema en inglés, `nextDouble()` rechaza `7,5` y se descartarían casi todas las filas. La línea de Sara termina en `;`: el `Scanner` no devuelve un campo vacío, sino que se queda sin campos y lanza `NoSuchElementException`; el `NP` de Hugo lanza `InputMismatchException`. Capturar la superclase cubre los dos casos. El acumulado por módulo solo suma alumnado válido porque las notas se suman **después** de leer los cinco campos sin error (si no, Hugo sumaría su 6 de AD antes de fallar en DI).

Solución Ejercicio 6: Migración y Fusión Ordenada de Padrones (`FusionSocios.java`)

```java
package soluciones_b1;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class FusionSocios {

    static int numero(String linea) {
        return Integer.parseInt(linea.split(";")[0]);      // "0012;..." -> 12
    }

    /** Cuenta los caracteres que no son ASCII (tildes, eñes...): en UTF-8 ocupan 2
bytes, en ISO-8859-1 uno. */
    static int noAscii(String linea) {
        int n = 0;
        for (char c : linea.toCharArray()) {
            if (c > 127) n++;
        }
        return n;
    }

    public static void main(String[] args) {
        Path antiguo = Path.of("datos", "socios_2009.txt");
        Path reciente = Path.of("datos", "socios_2014.txt");
```

```
        Path destino = Path.of("datos", "socios_unificado.csv");

        int leidos2009 = 0, leidos2014 = 0, escritos = 0, duplicados = 0,
caracteresNoAscii = 0;

        try (BufferedReader br1 = Files.newBufferedReader(antiguo,
StandardCharsets.ISO_8859_1);
             BufferedReader br2 = Files.newBufferedReader(reciente,
StandardCharsets.ISO_8859_1);
             BufferedWriter bw = Files.newBufferedWriter(destino,
StandardCharsets.UTF_8)) {

            bw.write("NUM;NOMBRE;LOCALIDAD");
            bw.newLine();

            // Una línea "en mano" de cada fichero: nunca hay más de dos en memoria
            String l1 = br1.readLine();
            String l2 = br2.readLine();

            while (l1 != null || l2 != null) {
                String elegida;
                if (l2 == null || (l1 != null && numero(l1) < numero(l2))) {
                    elegida = l1;                                   // el menor es el de
2009
                    leidos2009++;
                    l1 = br1.readLine();
                } else if (l1 == null || numero(l2) < numero(l1)) {
                    elegida = l2;                                   // el menor es el de
2014
                    leidos2014++;
                    l2 = br2.readLine();
                } else {                                            // mismo número: gana
el padrón de 2014
                    System.out.println("  Duplicado " + l1.split(";")[0] + ": se
conserva el dato de 2014 -> " + l2);
                    elegida = l2;
                    duplicados++;
                    leidos2009++;
                    leidos2014++;
                    l1 = br1.readLine();                            // se avanza en los
dos ficheros
                    l2 = br2.readLine();
                }
                bw.write(elegida);
                bw.newLine();
                escritos++;
                caracteresNoAscii += noAscii(elegida);
            }
        } catch (IOException e) {
            System.err.println("Error en la fusión: " + e.getMessage());
            return;
        }

        try {
            // Tamaños consultados DESPUÉS de cerrar el writer (si no, parte sigue en el
búfer)
            System.out.printf("Leídos: %d (2009) + %d (2014) · Duplicados: %d ·
Escritos: %d socios%n",
                    leidos2009, leidos2014, duplicados, escritos);
            System.out.printf("Tamaños: socios_2009.txt %d B + socios_2014.txt %d B
(ISO-8859-1) -> socios_unificado.csv %d B (UTF-8)%n",
```

```
                    Files.size(antiguo), Files.size(reciente), Files.size(destino));
            System.out.printf("Caracteres no ASCII escritos: %d (en UTF-8 ocupan %d
bytes; en ISO-8859-1 ocupaban %d)%n",
                    caracteresNoAscii, 2 * caracteresNoAscii, caracteresNoAscii);
        } catch (IOException e) {
            System.err.println("No se pudo consultar el tamaño: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola:

```
  Duplicado 0012: se conserva el dato de 2014 -> 0012;Ramón García Suárez;Avilés
  Duplicado 0021: se conserva el dato de 2014 -> 0021;Nuria Menéndez Ibáñez;Corvera
Leídos: 5 (2009) + 5 (2014) · Duplicados: 2 · Escritos: 8 socios
Tamaños: socios_2009.txt 169 B + socios_2014.txt 168 B (ISO-8859-1) ->
socios_unificado.csv 313 B (UTF-8)
Caracteres no ASCII escritos: 25 (en UTF-8 ocupan 50 bytes; en ISO-8859-1 ocupaban 25)
```

`datos/socios_unificado.csv` (UTF-8):

```
NUM;NOMBRE;LOCALIDAD
0003;Lorena Fernández Ruiz;Avilés
0005;Adrián Pérez Cueto;Illas
0007;Íñigo Muñiz Peña;Corvera
0012;Ramón García Suárez;Avilés
0015;Covadonga López Álvarez;Gozón
0018;María José Peláez;Soto del Barco
0021;Nuria Menéndez Ibáñez;Corvera
0030;Óscar Fernández Viña;Avilés
```

> **Notas para el aula:** es el algoritmo de *merge* de la ordenación externa: memoria constante sea cual sea el tamaño de los padrones, y funciona porque **las dos entradas ya vienen ordenadas** (si no, habría que ordenarlas antes). Las tres ramas del `if` cubren «solo queda o es menor la de 2009», «solo queda o es menor la de 2014» y «mismo número»; en el empate se avanza en los dos ficheros. Comprobación de tamaños: los dos padrones suman 337 bytes; al quitar las dos líneas duplicadas de 2009 (70 bytes) y añadir la cabecera (21) quedan 288 caracteres, y los 25 caracteres no ASCII ocupan un byte más en UTF-8: 288 + 25 = 313. Los `Files.size()` se piden fuera del `try` de escritura: dentro, parte del fichero seguiría en el búfer.